

repo-people App

Technical Overview - Tools, Technologies & Architecture

What is repo-people?

repo-people is a full-stack web application for exploring and analysing the people behind any GitHub repository. Given a repository owner and name, it fetches every relevant user - contributors, stargazers, forkers, watchers, issue authors, PR authors, and maintainers - and enriches each profile with location, company, follower count, account age, and more.

The results are presented in an interactive dashboard with sortable tables, charts, a geographic world map, role-overlap analysis, leaderboards, shareable links, and multi-format export (JSON, CSV, Markdown, PDF). Users can authenticate via GitHub OAuth to skip manual token management, or supply a Personal Access Token for higher rate limits.

Architecture Overview

The app is split into a React single-page application deployed on Vercel and a Python FastAPI backend deployed as a containerised service on Google Cloud Run. The two tiers communicate over HTTPS - REST for commands and Server-Sent Events (SSE) for live fetch progress streaming.

Component	Role	Purpose
Vercel (CDN)	Frontend static build (JS/CSS/HTML)	Frontend hosting
Google Cloud Run	Containerised Python service	Backend hosting
SQLite (aiosqlite)	Job results + OAuth sessions	Persistence
GitHub OAuth 2.0	User sign-in via popup flow	Authentication
GCP Secret Manager	OAuth client ID + secret	Secrets management
GCP Artifact Registry	Versioned Docker images	Image storage
Google Cloud Build	Build -> push -> deploy pipeline	CI/CD

Frontend Stack

React 18

TypeScript

Vite 5

Tailwind CSS 3

TanStack Table v8

Recharts

react-simple-maps

jsPDF

html2canvas

xlsx

Lucide Icons

Library	Category	Usage
React 18 + TypeScript	UI framework	Component-based SPA with full type safety
Vite 5	Build tool	Lightning-fast HMR in dev; optimised production bundles
Tailwind CSS 3	Styling	Utility-first CSS; dark theme throughout
TanStack Table v8	Data table	Virtualised, sortable, filterable user table
Recharts	Charts	Role distribution, account-age pie, leaderboard bars
react-simple-maps	World map	SVG geographic map of contributor locations

jsPDF + html2canvas

xlsx	Spreadsheet export	Client-side PDF generation from rendered DOM
Lucide React	Icons	CSV/XLSX file generation in the browser
Vitest + Testing Library	Testing	Consistent icon set across the UI
		Unit + component tests with DOM simulation

Backend Stack

Python 3.11

FastAPI

uvicorn

aiosqlite

PyGithub

httpx

sse-starlette

Pydantic v2

repo-people (PyPI)

python-dotenv

Library	Category	Usage
FastAPI	Web framework	Async REST API with automatic OpenAPI docs
uvicorn	ASGI server	Production-grade async server (ASGI)
aiosqlite	Database driver	Non-blocking SQLite for job + session persistence
PyGithub	GitHub client	Paginated GitHub REST API v3 calls
httpx	HTTP client	Async HTTP for OAuth token exchange
sse-starlette	Streaming	Server-Sent Events for live fetch progress
Pydantic v2	Validation	Request body validation and field coercion
repo-people	Data library	User role fetching and profile enrichment (PyPI pkg)
python-dotenv	Config	Local .env file loading for development
pytest + httpx	Testing	Backend API integration tests

Infrastructure & Deployment

The backend is containerised using Docker (python:3.11-slim base) and deployed to Google Cloud Run, which provides automatic HTTPS, scale-to-zero, and managed TLS. A single Cloud Run instance is enforced (maxScale=1) to avoid split SQLite state. The frontend is a static Vite build deployed to Vercel for global CDN delivery.

Tool	Category	Role
Docker	Container runtime	python:3.11-slim; builds via Dockerfile.cloudrun
Google Cloud Run	Backend hosting	Serverless containers; scale-to-zero; HTTPS
Vercel	Frontend hosting	Static CDN; automatic deploys from Git
GCP Artifact Registry	Image registry	Stores tagged backend Docker images
Google Cloud Build	CI/CD	Builds image, runs tests, deploys on push to main
GCP Secret Manager	Secrets	GITHUB_CLIENT_ID and GITHUB_CLIENT_SECRET
GitHub Actions	CI	Frontend lint, type-check, and test suite on PRs

Authentication Flow

GitHub OAuth 2.0 is used for user authentication. The flow:

1. Frontend opens a small popup to GET /auth/login on the backend.
2. Backend constructs a GitHub authorize URL (with state token for CSRF protection) and redirects the popup to GitHub.
3. GitHub redirects back to /auth/callback on the backend with a short-lived code.
4. Backend exchanges the code for a GitHub access token via httpx POST.
5. Backend fetches the user profile, creates a server-side session (aiosqlite), and sets a secure HttpOnly session cookie.
6. Backend redirects the popup back to the frontend with #auth=success in the URL.
7. Frontend detects the hash, closes the popup, and polls /auth/me to retrieve the authenticated user profile.

Key Features

* Multi-role fetching	Fetch Contributors, Stargazers, Forkers, Watchers, Issue Authors, PR Authors, Maintainers in parallel
* Live SSE streaming	Real-time fetch progress (users fetched, ETA, rate-limit remaining) via Server-Sent Events
* Rich analytics	Role distribution charts, account-age breakdowns, leaderboards, overlap analysis, health score
* Geographic map	World map of contributor locations using react-simple-maps + SVG
* Multi-repo compare	Fetch multiple repos and compare user overlap across jobs
* GitHub OAuth	Sign in once; backend uses your GitHub session for all subsequent fetches
* Shareable links	Generate 24-hour read-only share tokens for any job
* Multi-format export	Download results as JSON, CSV, Markdown table, or PDF report
* Job history	Persistent job list with tagging, renaming, and deletion
* Fetch cancellation	Stop an in-progress fetch at any time; partial results are preserved

Environment Variables

Variable	Scope	Purpose
VITE_API_BASE_URL	Frontend (Vercel)	Cloud Run backend URL; required for prod builds

BACKEND_URL

Backend (Cloud Run)

		Self-referencing URL used in OAuth redirect_uri
FRONTEND_URL	Backend (Cloud Run)	Vercel app URL; used in post-OAuth redirect
CORS_ORIGINS	Backend (Cloud Run)	Comma-separated allowed origins for CORS
GITHUB_CLIENT_ID	Backend (Cloud Run)	OAuth App client ID (via Secret Manager)
GITHUB_CLIENT_SECRET	Backend (Cloud Run)	OAuth App client secret (via Secret Manager)
FETCH_LIMIT	Backend (Cloud Run)	Max users per job (0 = unlimited)
REPO_PEOPLE_DB	Backend (Cloud Run)	SQLite DB path (default: /tmp/repo_people_jobs.db)

Testing

Tool	Layer	Purpose
Vitest	Frontend	Unit and component test runner (Vite-native)
@testing-library/react	Frontend	DOM rendering and user-event simulation
vitest-fetch-mock	Frontend	Mock fetch() calls in test suite
pytest	Backend	Python test runner for API integration tests
httpx (test client)	Backend	Async HTTP client used in pytest fixtures
GitHub Actions	CI	Runs full frontend + backend test suite on every PR